Road to Purple Team Glory

KC7 Series:Kusto Query Language 101

Image Generated by Google Gemini

At my new role, I have met some incredibly talented and chill individuals. One of said individuals was kind enough to entertain my request to "*TEACH ME YOUR WAYS*". So here we are, learning about Kusto Query Language (KQL) and doing blue team CTFs. The resource he gave me was KC7 Cyber, a learning academy with one of the most laid back yet highest quality learning resources I have seen for becoming a better hunter of *the enemy in the wire*.

Essentially, I just want to stick it to the bad guys.

Ahhhh you spent a year working out this meticulous plan to exploit my network, evade my controls, and pillage for loot? I am going to eat popcorn and watch you every step of the way until I feel like kicking you out - Somebody better than me probably

Starting out, let's learn a little bit about Kusto.

All credit to the KC7 team and their KQL 101 course.

First and foremost, KQL is a query language designed to interface with the Azure Data Explorer (ADX). Not going to lie, I didn't know what that was when I started this so let that be encouraging if you are the same teehee.

I went ahead and took the course on ADX here.

According to Microsoft,

Azure Data Explorer is a fully managed, high-performance, and big data analytics platform. Azure Data Explorer can take all this varied data, and then ingest, process, and store it. You can use Azure Data Explorer for near real-time queries and advanced analytics, as well as for more advanced features such as geospatial analytics, alerting, dashboarding, and business analytics.

...

Azure Data Explorer is a big data analytics platform that makes it easy to analyze high volumes of data in near real time. It allows you to extract key insights, spot patterns and trends, and create forecasting models.

Essentially, we want to make sense of vast amounts of data that flows in different formats.

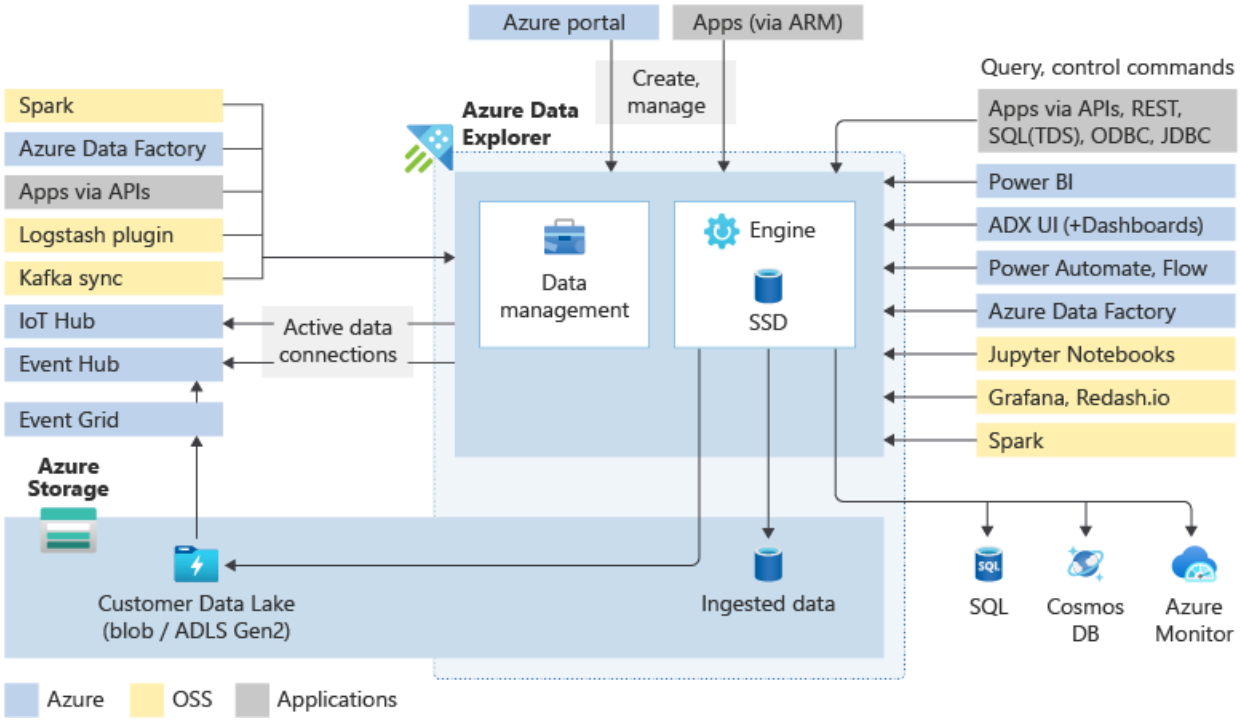


Image Credit to Microsoft Learn

ADX clusters are auto-scalable and cluster can hold up to 10,000 databases and each database up to 10,000 tables. Data per table is stored in shards called *extents.* Data is indexed and partitioned based on time, with a non relational database that is not constrained by primary foreign keys or uniqueness. This allows fast access to querying said data.

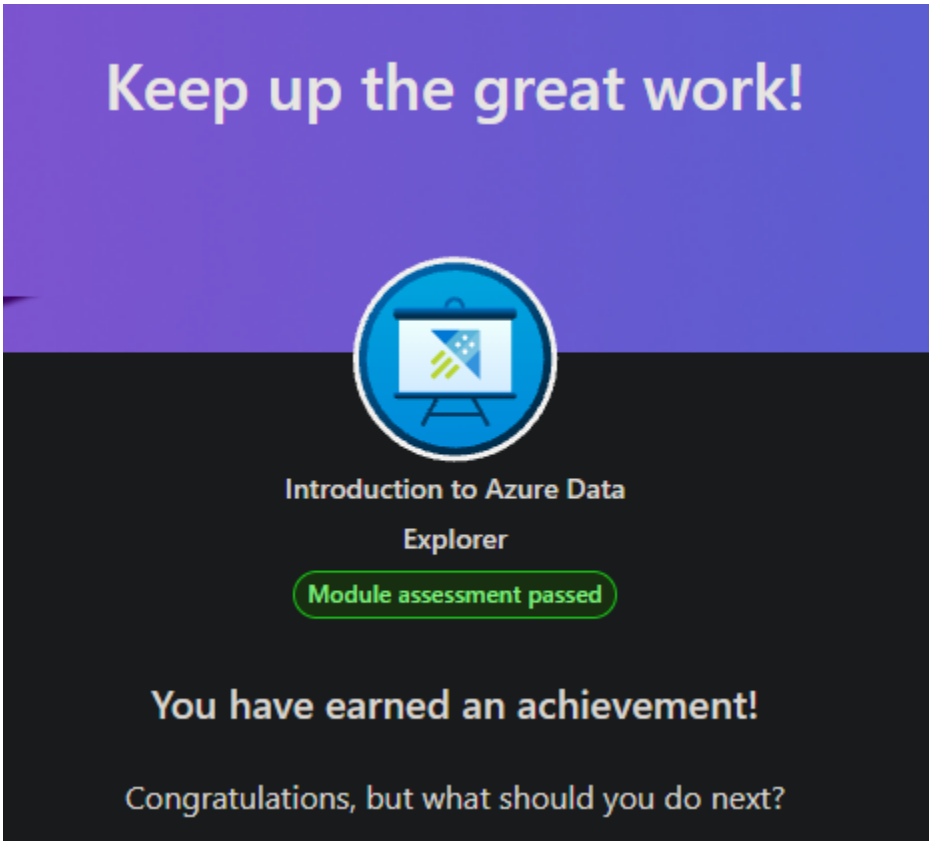
Interacting with ADX:

Ingest data into system => Analyze Data => Visualize Results

Ingestion loads the data into a table in ADX in batch or stream modes. Batching ingestion is optimized for high ingestion throughput and fast query results. Streaming ingestion allows near real-time latency for small sets of data per table.

The second phase in the above workflow is where Kusto Query Language shines brightest, and serves as the answer to the question '*How do I analyze my data?'.*

KQL supports cross-cluster and cross-database queries, and it's feature rich from a parsing (json, XML, etc.) perspective. In addition, the language natively supports advanced analytics.


Keep up the great work!
Introduction to Azure Data Explorer
Module assessment passed
You have earned an achievement!
Congratulations, but what should you do next?

I am something of an *Azure Data Explorer Expert* myself ;)

But in all seriousness, I recommend you to go through the course yourself just to get a bit of exposure. All we are looking to do is to build a foundation that we can continue to build on as we progress.

Now, back to KC7...

Essentially, the next section will be my notes with a bit of commentary.

Why is KQL important to the information security community specifically?

Security analysts, incident responders, threat intelligence analysts, and threat hunters use it every day to:

Search logs in seconds—because speed matters when investigating threats
Filter and analyze events—cut through the noise and focus on what's important
Identify anomalies and patterns—spot unusual activity before it becomes a bigger problem.

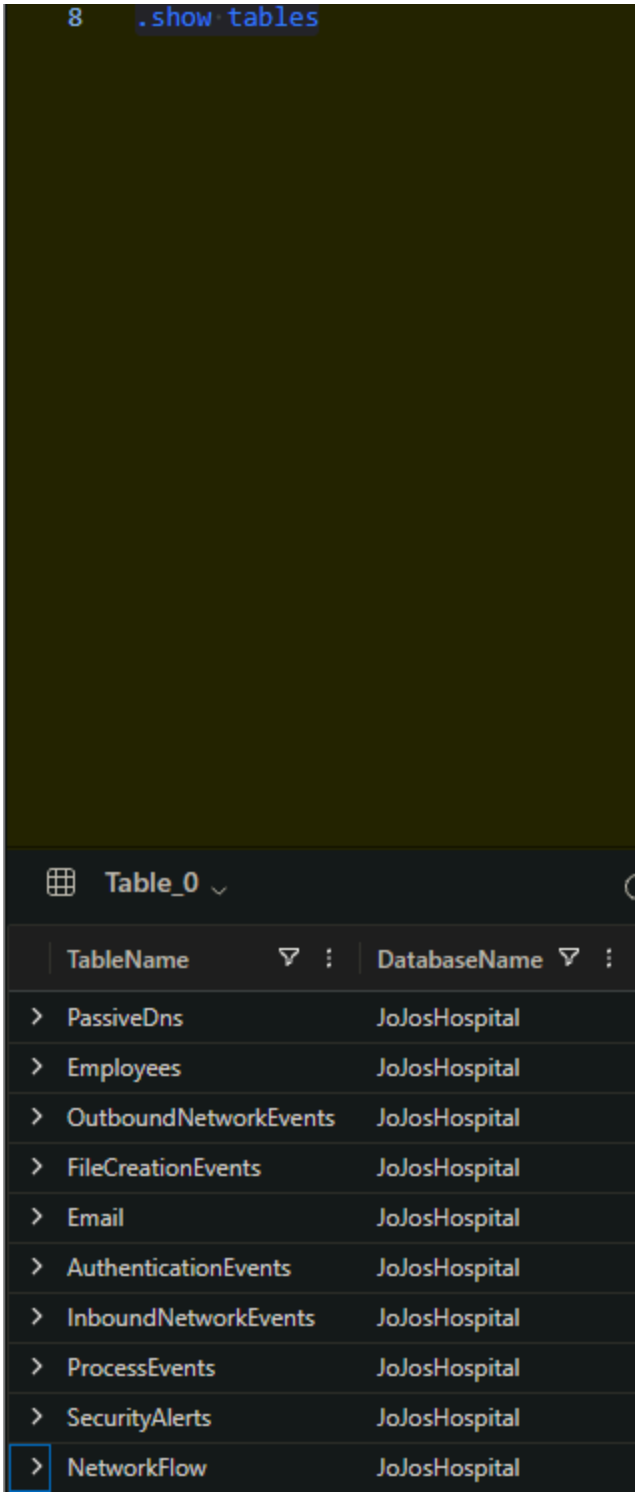
What problem does KQL solve?

Attackers, no matter how advanced, leave clues:

If you don't know how to query and analyze that data efficiently, those clues remain buried. KQL helps you connect the dots, so you can think like an investigator and stay ahead of cyber threats.

In KQL, rows are representative of individual records and columns store specific types of information such as time, user name, et cetera.

Like learning how to build queries on most other platforms, it is important to orient yourself with what tables are available, and what the data within looks like.



.show is an excellent utility to orient yourself with the database you are querying. Technically, it is a special command referred to as a control command or a management command, identified by the dot prior to the command name.

.show tables - shows you a list of tables
.show tables details - shows you the same but with more detailed output

.show database schema - shows you table names, columns names in each table, and the data type within each column

To get an idea of what the data in a particular table looks like we can use *take* to grab a random set of rows within the table. Take is a *key command.*

Let's try it with the SecurityAlerts table...

```
SecurityAlerts
| take 10
```

Count is another particularly useful command that gives us a quantitative representation of data. Say I wanted to see how many security alerts we have:

```
SecurityAlerts
| count
```

Distinct helps us remove duplicate data for less cluttered results. We can combine it with count to build a more concrete query, for instance:

```
Employees
| distinct role
| count
```

Where is an important command that is essential for narrowing down the data for precise results.

```
Employees
| where role == "Radiologist"
```

The double equal (==) operator gives exact matches. Also, if you are lazy like me, you may not want to see all of the columns in your result, just what you are looking for in particular. Say I want to find an employee's email address but not all of their other information contained in a row...

```
Employees
| where name == "Noemi Tep"
| project email_addr
```

Project-away can also be used to exclude a particular column if you want to see everything else, in the use case of the course I found the user-agent string to be a bit monotonous so...

```
Employees
| where name == "Noemi Tep"
| project-away user_agent
```

Contains and has are important key commands to search and gather matches. Say for finding emails with a particular word in the subject line.

The key difference is that contains finds *anywhere* that the word appears. So searching for health would match on 'healthcare' or 'healthy'. Has will match exactly on the word 'health', making it more efficient and thus less computationally expensive.

Something to note at this point, the questions in the CTF style format of KC7 are awesome because they reflect the internal questions you may have when performing an investigation.

For instance, I am wanting to build a query to find the answer to:

How many emails did we receive that have the word health in the subject?

Well, now I know a little bit about KQL and I can answer:

Where does this information live (in what table or database etc)?

In this case, likely the *Email* table. So we have the first part of the query.

Email
| ?

Next, what column or pieces of information are particularly of interest?
In this case, email subject lines where the word health is present

We now know that there is a column called *subject* based on our probing, and we know that the has and contains operators can be used to find patterns. In this case we want an exact match so we can use *has*.

Email
| where subject has “health”

Technically, this gives us the results we are looking for but *not as efficiently as it could be*. The last piece of the puzzle can be solved with the first part of the question we are trying to answer:

How many emails have we received that have the word health in the subject?

The answer to how many can be represented with *count*

Email
| where subject has “health”
| count

Bonkkkkkk! Success! *And* we worked through the thought process of building the query. Ws on top of Ws in this course!

We can use the ye old ‘!’ operator to exclude particular matches, IE *!contains <whatever_i_dont_want>*

Continuing on we see:

Who is the sender for the last external email received by Noemi?

The query provided is:

```
Email
| where recipient == "noemi_tep@jojoshospital.org"
| where sender !contains "jojoshospital"
```

But then, we have to do some flailing around in the output to get the answer...

Let's optimize it a bit for learning purposes:

We know we want to see the sender, that is the answer to our question, so I am thinking a *project sender* will be the last statement on the query. However, if we use that exclusively it is hard to get the answer because then all we see is *sender* and nothing else to be able to tell who the *last* sender is.

So what column is indicative of the answer for first, last, etc? Probably the one related to time. We can use *top* to sort by a column and order the output. In this case we want to see the last entry first, indicating a descending order. Also, we know the column we want to sort by is *timestamp*. Lastly, all we want to see is the very last email sent to Noemi, so we use *top 1.*

The final query looks like this:

```
Email
| where recipient == "noemi_tep@jojoshospital.org"
| where sender !contains "jojoshospital"
| top 1 by timestamp desc
| project sender
```

The *and* as well as *or* operators may be used intuitively to logically drill down results as well.

Now, how can we take the results from one query and use it to get results from another

table? The *let* statement allows you to declare a variable to store results to be referenced in a query into another table.

Note:
The *in* command lets you declare a list to be referenced as well, forgot to mention that earlier.

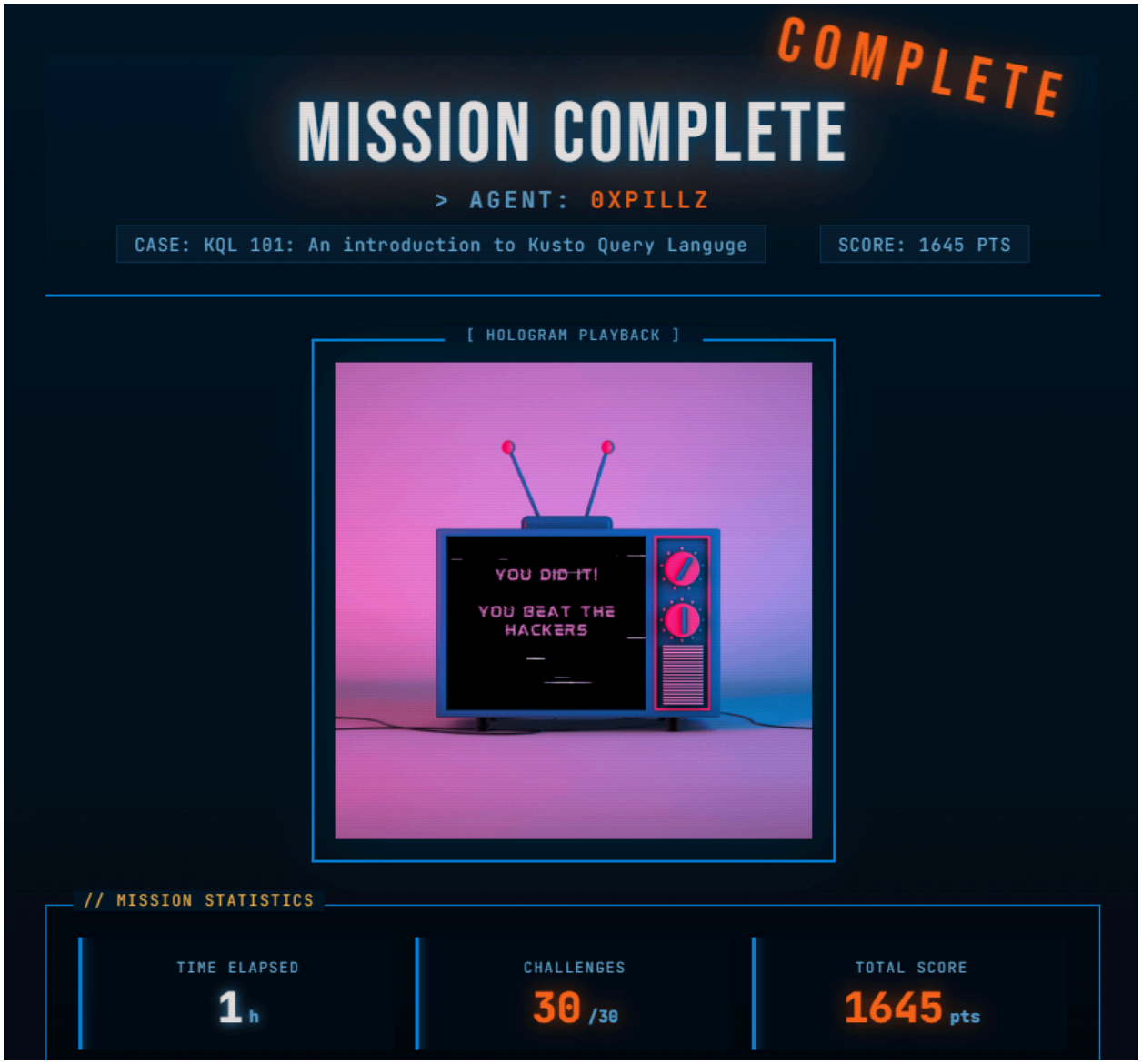
| where src_ip in ("10.10.0.144", "10.10.0.86", "10.10.0.86", "10.10.0.20", "10.10.0.109")

Take the following for example:

```
let mary_ips =
Employees
| where name has "Employee Name"
| distinct ip_addr;
OutboundNetworkEvents
| where src_ip in (mary_ips)
```

Look at the above disgusting conglomeration of colors. The blue part represents the variable declaration. This says *Let 'mary_ips' be a variable containing a result of the query in green, query A.* Then, query B highlighted in orange simply references the variable storing the result of query A to do its work and give the answer we want to see.

That wasn't so bad! The colors were the worst part ;)



We have become KQL masters ;)

Some more KQL resources:

https://www.youtube.com/@TenMinuteKQL
Book of KQL for Chads